Generating a Kubernetes Deployment Set from a Project-Intent Domain Model

Project Task 2: Model Transformations
Project Type: Code Generation
October 16, 2026

Joris Jonkers (S1829157)

Model Driven Engineering
University of Twente

Contents

1 Purpose of the Project **1**

1.1 Goals . . . 1

1.2 Role of the Transformations . . . 1

1.3 Refinement of Tasks 0 and 1 . . . 2

1.4 Assumptions and Scope . . . 2

2 Alternatives and Justification **3**

2.1 Existing Kubernetes Modelling Tools . . . 3

2.2 The Transformation Language . . . 4

2.3 The Structure of the Transformation . . . 4

2.3.1 Flattening Before the Resolution . . . 4

2.3.2 Resolving One Project with All Projects as Input . . . 4

2.3.3 Typed Extents . . . 5

2.3.4 Rejecting Unresolvable Inputs . . . 5

3 Solution Design **5**

3.1 The Flattening . . . 5

3.2 Interface, Mappings and Queries . . . 6

3.3 Worked Example . . . 7

3.4 The Derivation Rules . . . 8

3.4.1 The Process Mapping . . . 9

3.4.2 The Edge Rule . . . 10

3.4.3 Environment Variables and Placeholders . . . 10

3.4.4 Network Peers . . . 10

3.4.5 The Path Plan . . . 11

3.5 The Trace and End Sections . . . 11

3.6 The Black-Box Library . . . 11

3.7 The Final Pass . . . 11

3.8 Aborts and Running the Transformations . . . 12

4 Tests and Evaluation **12**

4.1 Results . . . 13

4.1.1 Revision Evidence . . . 13

4.1.2 Generated-File Evidence . . . 13

4.1.3 Gaps in the Evidence . . . 13

4.1.4 Defects Found . . . 14

4.2 Limitations . . . 14

4.3 Reflection . . . 15

A Changes to the Task 1 Metamodels **16**

A.1 The Pinned Inputs Metamodel 16
A.2 Changes to the Target Metamodel 16
A.3 Changes to the Source Metamodel 23
A.4 Grammars for the Pinned Inputs 23

B The Hand-in Archive **24**
B.1 Running the Transformations 25

1 Purpose of the Project

1.1 Goals

The project turns short deployment documents into Kubernetes files for an existing small cluster. Each project repository holds one `Project Intent` document describing its `Applications`: their `Processes`, memory and CPU limits, ports, dependencies and external routes. One `Platform Intent` document defines shared policies and resources, including cluster entry points, health-check timings and backup policies. Task 1 defined two Ecore metamodels: a source metamodel for the authored documents and a target metamodel for the `Resolved Deployment`. The target model records the values derived for each `Application` and the paths of the files to be generated.

Task 2 implements the transformation to `Resolved Deployment`. It combines all authored models with recorded cluster facts, image digests and other pinned inputs, producing one resolved model per `Project`. For example, an image name becomes a repository address pinned to a digest, and a dependency declaration becomes a host and port. Task 3 then generates files from these resolved values.

Term	Meaning
`Process`	One program of an `Application`, running in its own pods.
`Dependency edge`	A declaration by one `Process` that it calls a named port of another `Application`.
`Pinned input set`	The read-only inputs besides authored documents: node contract, images lock, `ClusterState` snapshot, migration proofs, `Assets`.
`Switchover`	How a new version replaces the old: blue/green (both run; traffic switches once the new is healthy), rolling, or stop-start.
`Release Gate`	The service that blocks a blue/green switch until every `Process` of the `Application` is healthy.
`Reconcile Unit`	A group of resources that Flux reconciles together, with dependencies on other such groups.
`Revision`	A release identifier computed by hashing an `Application`'s resolved configuration.
`Estate`	All `Projects`, the `Platform Intent` and the `Pinned input set` read by one run.
Flattened model	`Project Intent` with every shared declaration copied down onto the `Processes` that hold it (class `EffectiveProject`).

Table 1: Project Terms Used in This Report

The transformation must reproduce the oracle files supplied with the specification's examples: expected resolved values, dependency edges and generated files. If a value cannot be derived, the run must abort with an error message. A missing or fabricated value could otherwise produce a valid-looking file that deploys the wrong configuration.

The resolved model must also contain every value needed by the Task 3 templates, including paths, ports and names. Otherwise, tests of that model could pass while a template made an unchecked deployment decision.

1.2 Role of the Transformations

Figure 1 shows the pipeline. Xtext converts each document to a source model, and the Task 1 OCL constraints reject invalid ones. Three QVT-Operational transformations then run in sequence.

The first transformation, the flattening, copies declarations made once for an entire `Project` or `Application` down onto each `Process`. Its output is the flattened model, which still conforms to the source metamodel. Although it was implemented in Task 1, it was not reported there, so Section 3.1 covers it here. The second transformation, the resolution, reads the flattened models and the `Pinned input set`, the additional read-only inputs, and derives one complete `Resolved Deployment` per `Project`. It is the main transformation and the subject of this report. The third, the final pass, collects Vault policies from all resolved models to plan one job that installs them. This requires every project's output, so it runs after the individual resolutions (Section 3.7).

A small Java exporter writes the resolved dependency edges as JSON for comparison against the oracle. The Acceleo templates of Task 3 then read the `Resolved Deployments` and generate the Kubernetes files.

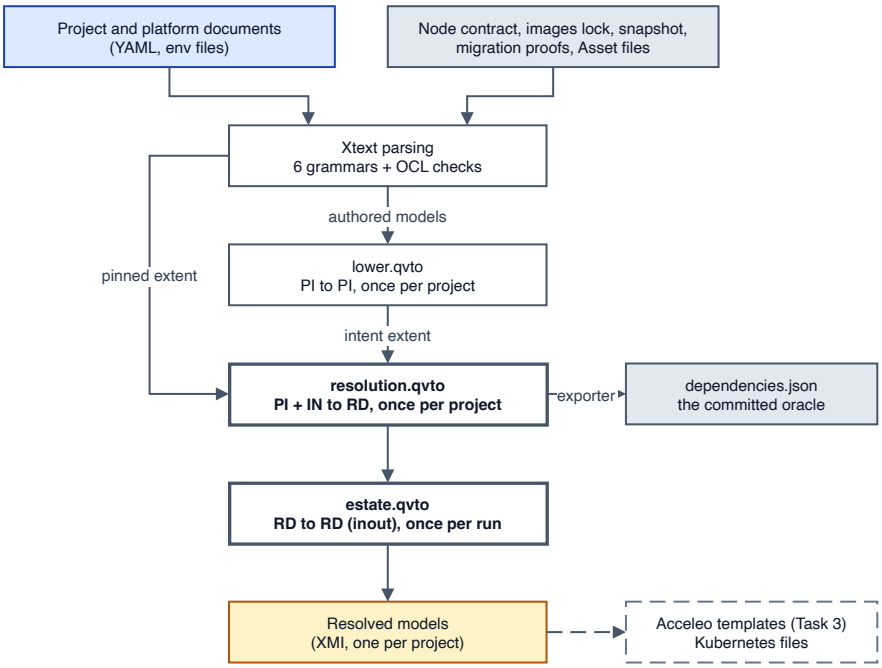


Figure 1: The Pipeline, With Task 2's Additions in Bold

1.3 Refinement of Tasks 0 and 1

Task 0 treated the node contract (the list of machines and their capacity), the images lock (each image pinned to a digest) and the cluster-state snapshot as plain inputs, while Task 1 only modelled the two authored documents. The resolution needs these as models, so a third metamodel now describes the `Pinned input set`, with its own Xtext grammar for each document (Appendix A.1). A separate package keeps these supporting inputs distinct from the deployment language used by `Project` authors.

Task 1 described three stages: authored, resolved and generated text. The flattening now makes inheritance explicit before resolution: each `Process` receives the declarations that apply to it from its `Project` and `Application`. Resolution rules therefore need not repeat the inheritance logic (Section 2.3.1).

Task 1 planned one resolved model for the entire cluster, because resolving dependency edges, routes and network access requires a cluster-wide view. Every run still reads every `Project`. The output is now one `Resolved Deployment` per `Project`, because each model then stays small and has one oracle to compare with (Section 2.3.2); the per-`Application` projection from Task 1 is still present as the `ResolvedApplication` inside that model. The final pass adds the one job that no single `Project`'s resolution can compute (Section 3.7).

Task 1's target metamodel only had a worked example for `minimal`. The other examples needed additional concepts: migrations, backup service accounts, API access, sidecar containers and twelve more file types. Appendix A lists every change.

1.4 Assumptions and Scope

The transformation assumes four conditions.

- **Valid input.** The Task 1 OCL constraints check the documents before transformation. Aborts reject invalid inputs that these constraints do not yet catch (Section 3.8).
- **One cluster per run.** Each execution of the pipeline reads all documents for one cluster, including one `Platform Intent` and one node contract that name the same cluster. All concepts, examples and oracles derive from that single existing cluster.
- **One reading of the specification.** If several providers satisfy a dependency, the transformation uses the deterministic name order recorded in the oracles. The comparisons check this interpretation; they do not establish that it is the only valid one.
- **No live cluster access.** The transformation only reads the documents provided to it. Any fact known

only to a running cluster, such as which node holds a particular volume, must arrive via the `ClusterState` snapshot captured by another process.

More Than One Cluster. The current implementation targets a single cluster, and every cluster-specific element is already a run input: the `Platform Intent`, the node contract and the `ClusterState` snapshot each name their cluster, and a `Project`'s environment files use a shared base plus a per-cluster overlay that the resolution selects via the `Platform Intent` cluster name (Section 3.4.3). Supporting a second cluster would require a separate run with that cluster's inputs and two extensions to the model: a `Project Intent` would need to declare which clusters it targets, and the network policy of a dependency edge to another cluster would need to allow an address range rather than a specific pod address. Such a dependency edge would resolve through the `Platform Intent` external provider list, as the current edge from `auth` to an external mail server already does (Section 3.4.2).

Table 2 sets the scope of Task 2.

In scope	Out of scope
The resolution of eight examples: `minimal`, `auth`, `data`, `delivery`, `edge`, `knowledge-platform`, `observability` and `secrets`.	The `knowledge` example, which models a project being retired (Section 4.2).
Every derivation these examples use, from dependency edges and placement to backups, migrations, network peers and the path of every generated file.	The text of the generated files, which is Task 3.
The flattening's extensions, the final pass, and the `Pinned input set` metamodel and grammars.	Applying the files to a cluster, and a second cluster.

Table 2: Scope of Task 2

2 Alternatives and Justification

2.1 Existing Kubernetes Modelling Tools

Three open-source tools were reviewed to see whether they could produce the `Resolved Deployment` from `Project Intent` and `Platform Intent`, or at least take part in the pipeline. Table 3 compares them.

Tool	What it does	Why it cannot replace the resolution	Where it could help
KubeDiagrams [1]	Draws diagrams from Kubernetes files or a live cluster, mapping kinds to icons.	It needs Kubernetes resources as input, so it cannot derive them from deployment intent.	It could visualize the generated files to show links produced by a `Project` document.
KubeOrch [2]	A drag-and-drop editor whose back end wires ports, fills templates, writes files and deploys.	It has no metamodel and no constraints; its rules are Go code, and it deploys directly, not via Flux.	Its graph notation could inform a `Project` document editor.
kubert [3]	A command-line tool to switch cluster contexts and run commands on multiple clusters.	It is an operations tool and does not model or generate anything.	It could apply files to several clusters, which is out of scope.

Table 3: Existing Kubernetes Modelling Tools

None of the three derives values from authored declarations. KubeOrch comes closest: it resolves a link between two services to an address and port, similarly to the dependency-edge lookup in this project. But it lacks a `Platform Intent`, node contract, images lock and cluster-state snapshot, so it cannot derive placement, image digests, backups or the connections permitted by network policies. It also provides no resolved intermediate

model to check before generating files. KubeDiagrams could visualize the Task 3 files so that dependencies and network connections could be inspected. A KubeOrch-style graphical editor could write `Project Intent` documents, since the editor and the transformation would work from the same Ecore metamodel.

2.2 The Transformation Language

Four languages were considered: ATL, QVT-Relations, QVT-Operational (QVTo) and Java with a rule library such as SiTra.

QVT-Relations was ruled out first. Its automatic traces and two-way synchronisation offer no benefit for a one-way derivation. In the cited experiments, one of its engines, medini QVT, ran into heap-space problems and became unresponsive on general-case input models with more than a hundred classes or a hundred attributes [5].

ATL, QVTo and Java were compared against the specific needs of this transformation. Most values in a `Resolved Deployment` do not derive from a single source element: a `Process`'s eligible machines depend on the node contract, a route's precedence on other routes, and a `Process`'s ingress peers on every `Process` in the cluster that depends on it. ATL's matched rules fit one source element producing one target element, with the engine choosing the order. Here, dependency lookup must search across `Projects`, and a `Revision` can be computed only after its `Application` has been assembled. ATL would still need helpers and explicit calls for these operations. Matched rules alone would therefore not remove the need to coordinate the transformation's execution.

A Java transformation could use standard Java debugging tools, but would require maintaining source-to-target references by hand. The derivations would be written as general-purpose code; participants in the cited discussion reported maintenance difficulties with model management in such code [4].

QVTo was chosen because these dependencies can be expressed directly: map the `Processes` before building routes that refer to them, then compute the `Revision` in an `end` section after the model is complete. Its trace lets a route find the resolved `Process` created from its source reference. OCL also allows the existing constraint vocabulary to be reused for lookups across the input models. Among the cited survey's respondents it was the second most-used transformation language after ATL, and its users named its maturity and its support by a standard as reasons to use it [4]. The one operation QVTo cannot express directly, computing a SHA-256 hash, uses QVT's black-box mechanism: a Java library called through a contextual operation (Section 3.6).

Speed was not a factor. The transformation is a proof of concept: it serves to work out the complete design and to check it against the specification's examples, not to run the cluster in the long term. The compiler is to be ported to a general-purpose language, such as TypeScript or, in the long term, Rust. Whether the cluster has enough capacity to host the applications is a separate limitation, discussed in Section 4.2.

2.3 The Structure of the Transformation

Every derived value must be recorded in a model before code generation. This allows tests to compare the values with the oracles independently of the templates that write them to files.

2.3.1 Flattening Before the Resolution

A single transformation from authored model to `Resolved Deployment` would work, but a flattening precedes the resolution. The flattening copies shared declarations down to each `Process`. For example, a `Project` startup budget of 30s reaches every `Process` except one that declares a more specific value, such as 20s. Resolution can then read the selected budget directly from each `Process`, without searching its ancestors. The chain also matches QVT specification Section 8.1.13, which composes a preparation transformation before the main one [6].

2.3.2 Resolving One Project with All Projects as Input

One resolution over all `Projects` at once could produce one `Resolved Deployment` for the whole estate. Instead, each invocation resolves one `Project` into its own model while reading every `Project`'s documents from the same input extent. Each model stays small and can be compared with its own oracle, while the shared extent provides the cluster-wide view needed to resolve dependencies, routes and network access. One value, the digest

over all Vault documents, needs the output of every `Project`, so the final pass, which runs over all resolved models together, adds it (Section 3.7).

2.3.3 Typed Extents

A rule also has to find the run's other documents. Using OCL's `allInstances()` would search the loaded models on each evaluation, making the search scope depend on which files had been loaded. Instead, the transformation selects each input once by type from two extents that hold exactly the run's documents, and every rule navigates from one of these properties (Section 3.2).

2.3.4 Rejecting Unresolvable Inputs

A default for a value the resolution cannot derive would give a file that looks valid and deploys the wrong configuration, so the run aborts instead, through a fatal assertion in an abort query (Section 3.8). The caller then has to handle the failure.

3 Solution Design

Task 2 uses three QVT-Operational transformations. The flattening (`lower.qvto`) is about 340 lines with 37 mapping operations and 8 queries. The resolution (`resolution.qvto`), the main transformation, is about 1700 lines with 8 mapping operations and 140 queries. The final pass (`estate.qvto`) is about 130 lines with no mappings but one helper.

Figure 2 shows the metamodels each transformation reads and writes. The three differ in kind, and all are one-way.

- The flattening is endogenous and horizontal: it reads and writes the source metamodel at the same abstraction level, producing a new model. One `Project` goes in, one comes out.
- The resolution is exogenous and vertical, a refinement: it reads source and pinned-input metamodels and writes the target metamodel, taking several input models and producing one output.
- The final pass is endogenous and in-place: it updates the resolved models of a run through an `inout` extent.

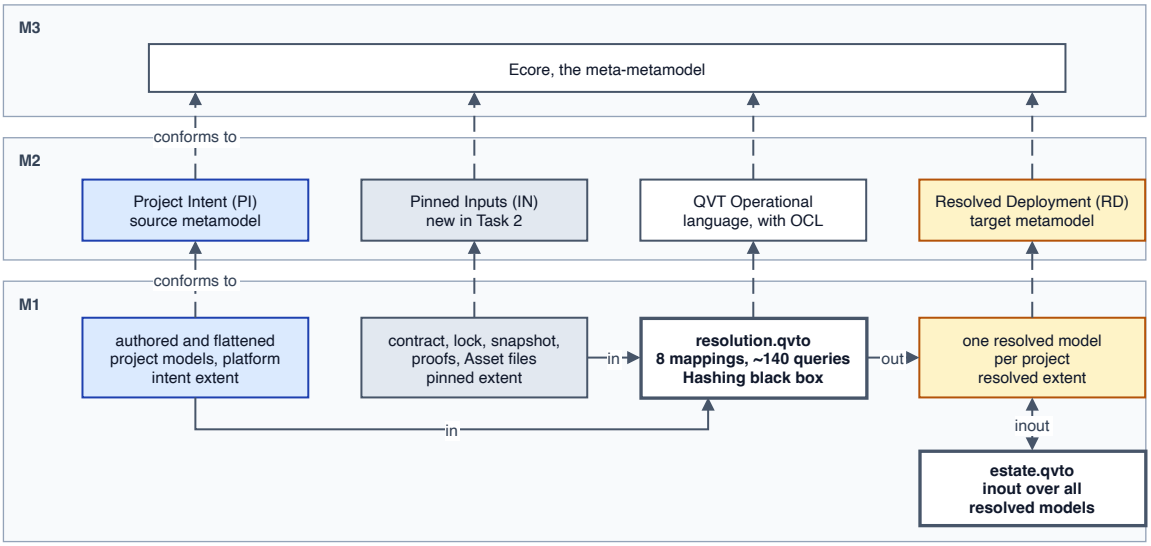


Figure 2: The Resolution and the Final Pass in the Metamodel-Based Transformation Pattern

3.1 The Flattening

The flattening, `lower.qvto`, turns each authored `Project` into an `EffectiveProject`. Every `Process` carries all shared declarations that apply to it, while the `Project` and `Application` carry none. Task 2 extended the

Task 1 implementation with API access and engine `Grants` (declared access to a secret of a database or transit engine).

Its 37 mappings are nearly all simple copies; its queries do the merging. A list such as `secrets`, dependency edges, configuration files or environment entries becomes the union of declarations from `Project`, `Application` and `Process` levels. If several levels declare the same item, the most specific declaration wins. A single value, such as the startup budget or cutover (whether the `Application` keeps serving during an update), also comes from the most specific level that declares it. Memory and CPU remain only on the `Process`.

A shared declaration needs a separate copy in each receiving `Process`. Reusing a cached mapping result would move that object to the last `Process`, because an EMF object can have only one container. Passing the receiving `Process` to `copyFor(self)` makes the cache distinguish these copies. Migration copying uses a disjuncts mapping to preserve whether the source declares a managed migration, self-managed migration or no migration.

The flattening has no aborts. The OCL constraints run before it, so every input it sees is valid.

3.2 Interface, Mappings and Queries

Example 1 shows how the inputs are grouped: `intent` contains the deployment declarations, `pinned` contains their supporting inputs, and `resolved` receives the target model.

```
transformation resolution(in intent : PI, in pinned : IN, out resolved : RD)
        access library Hashing;
configuration property project : String;    -- the project this run resolves
property estate : Platform = intent.rootObjects()[Platform]->any(true);
property projects : Sequence(EffectiveProject) =
        intent.rootObjects()[EffectiveProject]->sortedBy(p | p.project)->asSequence();
property lock : ImagesLock = pinned.rootObjects()[ImagesLock]->any(true);
-- and likewise contract, snapshot, proofs and assetFiles
main() {
        projects->select(p | p.project = project)->any(true).map deployment();
}
```

Example 1: The Signature and Inputs of the Resolution, Abridged

The intent extent holds, for every `Project` in the run, its authored model and its `EffectiveProject`, plus the `Platform Intent`. Authored models remain only because `Platform Intent` references them via Ecore cross-references. Every rule reads the `EffectiveProject` models.

The pinned extent holds the node contract, images lock, cluster-state snapshot, each `Project`'s migration proof (the schema version its migration was tested against), and one `AssetFile` per configuration file. Two configuration properties supply the `Project` name to resolve and the schema package digest to include in the provenance record.

Properties such as `projects` and `lock` select these inputs once. A dependency lookup navigates through `projects.applications.processes`, so it searches only the flattened models supplied to this run. Sorting `Projects` by name makes the result independent of input file loading order.

Mappings are used for objects that other rules must reference. For example, the trace connects a source `Process` to its `ResolvedProcess`, allowing a route to reuse that result through `resolveone`. A probe belongs only to its parent and needs no such lookup, so a query creates it with an object expression instead. Each mapped target type has one mapping, making the source and result type sufficient to identify the trace entry. Callers filter elements with `->select` before invoking a mapping. No mapping uses a `when` guard or inherits from another.

Table 4 lists the eight mappings and their purpose.

Queries calculate values without modifying the input models. They return scalars such as deadlines, new contained objects such as backup plans, or the target spelling of an enumeration value, for example `businessHours` as `business-hours`. The calling mapping assigns the result. Abort queries terminate the run when a value cannot be derived (Section 3.8). The final pass uses a helper instead because it adds files to an existing output model.

Mappings run in a fixed order. The deployment mapping creates `ReconcileUnits` first, then `Applications`,

Mapping	Creates, and why it is a mapping
`EffectiveProject::deployment`	The `ResolvedDeployment`. Its end section sets values that need the complete model.
`EffectiveProject::unit`	The `Project`'s `ReconcileUnit`, the group Flux applies together. Each `Application` refers to it.
`EffectiveProject::provenance`	The digest of every input and the hash over them. Its end section computes the hash.
`EffectiveApplication::application`	A `ResolvedApplication`. Generated files and the `Revision` refer to it.
`EffectiveApplication::gate`	The `ReleaseGate`'s inputs. The `Application` and its configuration refer to them.
`EffectiveApplication::migrationOf`	The database migration. Its network policy refers to it.
`Process::process`	A `ResolvedProcess`. Gate members, routes, scrapes, policies and generated files refer to it.
`Exposure::exposure`	A `ResolvedExposure`. The generated route file refers to it.

Table 4: Mapping Operations of the Resolution

provenance and the path plan, the step that gives every generated file its path. Each `Application` maps its `Process`es before its `Exposure`s (the routes that reach them from outside), `ReleaseGate` and migration, because those refer to `Process`es through the trace.

3.3 Worked Example

The `minimal` example declares one `Application`, notes, with one `Process`, notes-api, in about 35 lines of YAML. Table 5 follows declarations through to derived values.

Declared	Rule	Derived in the resolved model
`image: notes-api`	`locked`	`ghcr.io/jorisjonkers-dev/notes/notes-api@sha256:9c1e...`, user/group 1000, from images lock
`startupBudget: 20s`	`deadlineOf`, startup probe	deadline 60s (three budgets); the startup probe polls the liveness target every 5s and fails after 4 attempts
`cutover: continuous`	`switchoverOf`, `gate`	blue-green; `Release Gate` with 1 member, deadline 60s, cadence from the `Platform Intent`, error/latency checks
`memory: 256Mi, cpu: 50m`	`eligible`	all 7 machines in the node contract, each tested for twice the request size for blue/green deployment
`audience: anonymous`	`tierFor`, `exposure`	entry point public-frankfurt with TLS or ACME certificates, proxy traefik-public in the `edge-system` namespace
route /, prefix	`precedenceOf`, `middlewareOf`	precedence 1; security headers with strict content policy
`runtime: node`	`profileOf`	environment variables `OTEL_SERVICE_NAME`, `PORT=8080`, `DEPLOYMENT_ENVIRONMENT`, collector address from observability
scrape on http	`ingressOf`, `monitorOf`	ingress from traefik-public and prometheus on port 8080; pod monitor for blue/green
(nothing)	path plan	12 generated-file entries (`Deliverable`), including `apps/notes/namespace.yaml` and `estate/edge/public-frankfurt/notes-public.yaml`

Table 5: Declarations of `minimal`, Followed Through the Resolution

Four values depend on other run documents: the image needs the images lock, eligible machines need the node contract, the proxy needs the `edge Project`, and the collector address needs the observability `Project`.

Example 2 shows the authored `Process` in YAML, and Example 3 shows the `Process` the resolution assembled from it, in XMI.

```
- name: notes-api
  image: notes-api
  placement: { memory: 256Mi, cpu: 50m }
  startupBudget: 20s
  cutover: continuous
```

Example 2: The Authored `Process` notes-api, in YAML (Condensed)

```
<processes name="notes-api" deadline="60s" switchover="blue-green"
    image="ghcr.io/jorisjonkers-dev/notes/notes-api@sha256:9c1e5a4b..."
    uid="1000" gid="1000" memory="256Mi" cpu="50m">
  <startup period="5s" timeout="5s" failures="4"> ... </startup>
  <placement> <eligibleNodes>enschede-t1000-1</eligibleNodes> ... </placement>
</processes>
```

Example 3: The Resolved `Process` notes-api, in XMI (Abridged)

The query `deadlineOf` derives a 60s deadline from the declared `startupBudget: 20s`; the startup probe is recorded alongside it (Example 3). The query `switchoverOf` derives blue/green replacement from `cutover: continuous`. The path plan assigns the corresponding Flagger `Canary` resource to `apps/notes/notes/canary.yaml`, one of the twelve `Deliverable` entries in Table 5. The `gate` rule derives the `Release Gate` inputs, which a configuration map supplies to the service that holds the switch until every `Process` is healthy. Section 4.1 checks this whole model against the complete expected model of `minimal`, attribute by attribute.

3.4 The Derivation Rules

Table 6 lists the rules by target element. Each implements a derivation from the specification.

Target value	Rule	Derivation
Application		
reconcile unit and order	`unit`, `afterOf`	One unit per `Project`. Flux must wait for the units of its dependency providers, and for the secrets unit when secrets are needed.
Secret Store	`synced`, `storeEndpoint`	The Secret Store (Vault) address used when a secret, backup or migration needs it.
migration	`migrationOf`, `datastoreOf`	Combine the locked runner image, schema version from the migration proof, and database address from dependencies; derive its credential and permitted connections.
release gate	`gate`, `switchesBlueGreen`	Register each blue/green `Process` with readiness as a gate member. Use the largest member deadline and platform analysis settings; enable error-rate and latency checks for members exporting telemetry.
Process		
image, user, group	`locked`	The lock entry for the image name.
switchover	`switchoverOf`	stop-start for a `Process` whose `Application` may stop serving during an update, rolling for the platform's own delivery `Applications` (the `Release Gate` and the Reporter), blue/green otherwise.
deadline	`deadlineOf`	Multiply the startup budget by three and round down to whole seconds. A preparation job gets its budget rounded up instead; without a budget, use 600s.

Target value	Rule	Derivation
probes	`Probe::cadenced`	Declared target with the period, timeout and failures from the `Platform Intent`. The startup probe polls the liveness target every 5s until the budget runs out.
eligible machines	`eligible`	Keep nodes meeting the declared placement requirements and CPU/memory requests, including sidecars. Continuous replacement requires capacity for both old and new copies.
environment	`environmentOf`, `entryFor`	Cluster-specific entries replace base entries of the same name. Resolve placeholders, add runtime-profile variables, and sort by variable name.
secrets	`Grant::resolvedFor`	Key-value grant keeps the path and keys and gets a Kubernetes Secret. A transit or database grant becomes the Vault paths its policy covers.
volumes and backups	`backupFor`, `backupEgress`	For recoverable and irreplaceable volumes: schedule and retention from the `Platform Intent`, backup image for engine, backup service account and claim, connection, credential, off-cluster copy.
configuration files	`Asset::resolvedFor`	The file text, named after the `Process`, the file and a digest prefix, so a changed file gets a new name.
dependency edges	`edgeOf`	The first `Process` in the cluster with the `Application` and port the dependency edge names, with address and network peer; otherwise external providers.
Exposure		
entry point	`tierFor`	Select the first platform entry point whose allowed audiences include the `Exposure`'s declared audience.
route precedence	`rankOf`, `precedenceOf`	Exact routes come before prefix routes, and longer prefixes before shorter ones.
middleware	`middlewareOf`	Authentication for authenticated routes, security headers, and redirect if declared.
Network policy, per Process		
ingress	`ingressOf`	Proxy of each route entry point, metrics collector if the `Process` is scraped, every `Process` with dependency edge to its ports, its backup service account, migrations of databases it holds, delivery events receiver.
egress	`egressOf`	Cluster DNS, and the Secret Store if the `Process` holds a secret.
Deployment		
provenance	`provenance`	Record each input's digest, then hash that record to identify the complete input set.
revision	end of deployment	Digest of each `Application`'s complete element (Section 3.6).
generated files	`projectFiles`, `applicationFiles`, `edgeFiles`, `vaultFiles`	Path of every file, with elements it is generated from (Section 3.4.5).

Table 6: Derivation Rules, by Target Element

3.4.1 The Process Mapping

Example 4 shows how the `Process` mapping combines these rules. The image lock entry supplies the repository and digest; `deadlineOf` supplies the deadline; `eligible` filters the nodes. Optional fields are assigned only where they apply. For example, a one-off job has no switchover: assigning the enumeration's default value would incorrectly record a blue/green deployment strategy for that job.

```
mapping PI::Process::process(in app : EffectiveApplication, in prj : EffectiveProject)
```

```
                : ResolvedProcess {
        init { var own : LockedImage := locked(self.image); }
        name := self.name;
        image := own.repository + '@' + own.digest;
        if self.lifecycle = Lifecycle::application
                then switchover := switchoverOf(app, self) endif;
        deadline := inSeconds(deadlineOf(self));
        secrets := self.secrets->collect(g | g.resolvedFor(self));
        placement := object ResolvedPlacement {
                eligibleNodes := contract.nodes->select(n | eligible(self, n))->collect(n
    | n.name);
        };
        dependencies := self.dependsOn->collect(e | edgeOf(e));
        environment := environmentOf(self, prj)->sortedBy(e | e.name);
}
```

Example 4: The Process Mapping, Abridged

3.4.2 The Edge Rule

A dependency edge names an `Application` and a `Surface` (named port). For example, an edge to `platform-postgres` names its `postgres` port. The query `edgeOf` finds the first `Process` in the cluster belonging to that `Application` that provides that `Surface`. The resolved edge records the `Process` address and the network peer the consumer may reach. If no `Process` provides the `Surface`, the `Platform Intent` external provider list is tried; this is how `auth` reaches a mail server outside the cluster. If neither has it, the run aborts.

For a `Project` still running at its old location, the handover ledger in the `Platform Intent` supplies that location's namespace and name. The resolved address and network peer use these values, allowing consumers to reach the `Project` before it moves.

3.4.3 Environment Variables and Placeholders

A `Process`'s environment files consist of a base file and, optionally, one overlay per cluster. The resolution keeps base entries not replaced by the overlay for the `Platform Intent` cluster.

An entry may contain a placeholder, such as `DB_HOST=${dependency:platform-postgres.host}`. The query `Placeholder::entryFor` handles four kinds. A secret placeholder becomes a reference to a key of a secret the `Process` holds, so the value never appears in a file. A dependency placeholder takes the host or port of the `Process` dependency edge from `edgeOf`. An exposure placeholder takes the URL, host or scheme of an `Exposure` anywhere in the cluster. An identity placeholder takes the `Process` name, namespace or Vault role. Text after a placeholder is kept, as in `AUTH_LOGIN_URL=${exposure:auth.public#url}/login`.

The runtime profile then adds the platform-defined variables for the selected language runtime, such as `PORT`. Declaring one of these names in an environment file aborts the run, because it would conflict with the platform-defined value.

3.4.4 Network Peers

The ingress policy of a `Process` combines peers from six sources (Table 6). Most require looking across `Projects`: consumers with a dependency edge to the `Process`, migrations that access its database, proxies and the metrics collector named by the `Platform Intent`. Each source yields tuples of rule, namespace, `Process` and port. Distinct `PolicyPeer` objects would not compare equal merely because their fields match. The union therefore uses tuples and `asOrderedSet()` to remove duplicates before creating `PolicyPeer` objects. The rule name remains part of each tuple, so two rules allowing the same connection retain both rule identifiers.

3.4.5 The Path Plan

The path plan represents each generated file as an instance of a `Deliverable` subclass, recording its path and the model elements used to generate it. Each `Project` gets a namespace file, index and default-deny network policy. It also gets a secret operator connection if an `Application` synchronises secrets.

Each `Application` gets a directory with one file per resource kind: a workload file, a service account, RBAC, volume claims, a configuration map carrying `Release Gate` inputs as JSON, backups, a migration, a monitor, synced secrets, and a network-policy file with one policy per `Process`. Blue/green `Processes` add a `Canary` file, others a `Service` file; multi-replica `Processes` add a disruption budget. Empty files are not planned.

Each index lists the files Flux should apply from its directory. Network policies are included in the generated output for review, but excluded from these indexes while the specification's policy stage is render-only. This allows their proposed connections to be inspected before enforcement. Each `Exposure` gets a route file under its entry point (Traefik listener). Each service account holding a secret gets a Vault policy and role.

Because paths are in the model, two files claiming one path are caught before any text is written; no path is chosen inside a Task 3 template.

3.5 The Trace and End Sections

Gate members, routes, metrics scrapes and per-`Process` files refer to a `ResolvedProcess`. They find it by calling `p.resolveone(ResolvedProcess)` on the source `Process`, which the `Application` mapping has already transformed (Section 3.2).

The `end` sections set values whose prerequisites are created by the mapping body. The `Revision` must wait until the `Application` is complete; the migration job tag then takes a prefix of that digest so both identify the same release. Similarly, the render hash is computed after all input digests have been recorded, and `ReconcileUnit` dependency references are assigned after their target units exist.

3.6 The Black-Box Library

A digest is a SHA-256 hash over canonical text. OCL lacks a byte-hashing operation, so the Java class `Hashing` provides three contextual operations. `digestOf` walks any model element reflectively, writes canonical JSON and hashes it. `textDigestOf` hashes a file's text, for the content-derived names of configuration files. `rawDigestOf` hashes a string as-is.

The `Revision` uses `rawDigestOf`. Generated files carry this digest, so it must match the oracle exactly. The specification hashes the `Application` element in canonical JSON (RFC 8785), but its resolved document has a different structure from the target metamodel. The OCL query `elementJson` therefore constructs the required JSON from the source and target models, sorting keys, omitting absent values and using the specified representation of each value. The black box only hashes this text; the transformation determines which values it contains.

The QVTo compiler does not recognise a model type as conforming to `EObject` when looking up a contextual operation. Each call to an operation declared on `EObject` therefore needs an explicit cast: `p.oclAsType(ECORE::EObject).digestOf()`. The library unit also needs a name different from its Java package, because the package contains the QVTo keyword `resolve`. The standalone runner and Eclipse's `plugin.xml` register it as `dev.jorisjonkers.deploykit.emf.hashing`.

3.7 The Final Pass

One job installs the Vault policies and roles from every `Project` into the Secret Store. Its name contains a digest of all these documents. Changing a document therefore gives the next deployment a new job name, causing the updated configuration to be installed.

The final pass, `estate(inout rendered : RD)`, runs once over all `Resolved Deployments` produced together. It collects every `VaultPolicyFile` and `VaultRoleFile`, writes each as canonical JSON in OCL, sorts them by file name, hashes the whole with `rawDigestOf`, and plans the job, configuration map, service account and network policy. These files and their index are stored in the first model, so the job is generated once for the combined output. The pass also plans one cluster-wide route forwarding Flux events to the Reporter. Conflicting job settings abort the pass: all projects in the run must use the same configuration from their shared `Platform Intent`.

3.8 Aborts and Running the Transformations

Fifteen fatal assertions in the resolution abort the run when a value cannot be derived. Thirteen are in abort queries that a rule calls instead of returning an invalid value. For example, `locked` returns the lock entry for an image or calls `unlocked`, whose assert fatal ends the run with `E_UNLOCKED_IMAGE`. Two check `Release Gate` preconditions at the start of the gate mapping. The final pass has one more, which aborts it when two `Projects` carry different Vault policy jobs.

These assertions cover missing or unusable inputs, unresolved references and declarations that cannot be honoured. Examples include an image absent from the lock, a missing configuration file, an unsupported quantity unit, a dependency with no provider or a migration with no database. Other checks reject placeholders in configuration files, references to secrets the `Process` does not hold, author-defined runtime-profile variables, backed-up `Volumes` without a schedule or database engine, blue/green `Applications` without readiness, a `Release Gate` or delivery policy, and `Grants` without a Secret Store.

Where the specification defines an error code, the message contains it. An audience unsupported by any entry point needs no abort here: the `E_NO_TIER_FOR_AUDIENCE` constraint already rejects it before the transformation runs.

Three Java classes (`Lowering`, `Resolution` and `Estate`) run the transformations through QVTo's own `TransformationExecutor`. They register the black box, set configuration properties, pass the extents, and return the result or fail with the executor's diagnostic.

The build executes the transformations with QVTo's standalone executor on every code change; Eclipse launch configurations can also run them (Appendix B). The pipeline builds the extents from parsed and checked documents and writes every example's models under `target/parity/<example>/`. Appendix B.1 gives the steps.

4 Tests and Evaluation

The transformations are tested by running them on eight examples and comparing their output with oracle files: the expected outputs committed with the specification's worked examples. Matching them shows the transformation derives the values the specification expects.

The build runs six test types on every change.

- **Flattened models.** For the four examples with shared declarations, the flattened model is written as JSON and compared byte-for-byte with the expected flattened model.
- **Dependency edges.** For each example, the resolved edges are exported as JSON and compared byte-for-byte with the expected file. These tests read only files the pipeline wrote, so they check the pipeline as a user would run it.
- **Revisions.** Each expected resolved file records its `Revision`, a digest of the `Application` element in the specification's canonical JSON form. A matching `Revision` checks the values included in that representation (Section 4.1.1). The digest covers the element's own decisions, not the whole resolved model: the path plan, the network policies and the Vault documents fall outside it, which is what the complete-model and generated-file comparisons below are for.
- **Complete expected models.** `minimal` and `secrets` have full expected `Resolved Deployment` models, compared using EMF's structural equality (`EcoreUtil.equals`), which checks every attribute and reference.
- **Generated files.** Tests run the Task 3 templates on the transformation output and compare every generated file with its committed counterpart. This checks the resolved values used by the templates as well as their serialization.
- **Interface tests.** JUnit tests check that the resolution accepts a minimal `Project`, rejects a `Project` absent from the input, and reports the file name when a configuration file is missing. Final-pass tests check that one Vault policy job covers two `Projects`, its name changes when a document changes, no job is generated without documents, and conflicting job definitions abort the run. Eight flattening cases check the merging of shared declarations.

The coverage and mutation tools measure the Java integration code, which must reach full line and branch coverage and a 100% mutation score. They cannot measure QVTo. The checks above therefore provide functional evidence for the transformation rules, without establishing QVTo branch coverage.

4.1 Results

Table 7 lists the eight examples. All resolve successfully and every comparison passes, including the complete-model comparison of `minimal` and `secrets`. The full build runs 323 tests, 81 of them comparisons with the expected output, and passes.

	Counted in the resolved model							Equal to the expected output			
Example	**Apps**	**Proc.**	**Edges**	**Secr.**	**Vol.**	**Env.**	**Files**	**Edges**	**Revisions**	**Files**	**Model**
`minimal`	1	1	0	0	0	6	12	yes	1/1	yes	yes
`auth`	1	2	4	3	0	21	19	yes	1/1	yes	–
`data`	3	3	0	1	3	3	37	yes	3/3	yes	–
`delivery`	4	4	1	0	0	0	27	yes	4/4	yes	–
`edge`	2	2	0	0	0	0	13	yes	–	–	–
`knowledge-platform`	4	7	5	4	6	84	49	yes	1/1	yes	–
`observability`	2	2	0	0	0	0	15	yes	–	–	–
`secrets`	1	1	0	0	0	0	8	yes	–	–	yes

Table 7: The Resolved Examples and Their Comparisons (–: no committed oracle of that kind)

The `secrets` example has no JSON oracle for its resolved values and no committed generated files. It does have a complete expected EMF model, which checks every attribute and reference in the transformation output.

Between them, the examples exercise every row of Table 6. `minimal` exercises the common `Process` and `Exposure` rules (Table 5). The migration, placeholder and external-provider rules run in `auth`, which holds three secrets and reaches a mail server outside the cluster. The volume, backup and configuration-file rules run in `data`, which holds one volume of each durability class (reconstructible, recoverable, irreplaceable), a `Sidecar` and a configuration file. The rolling and stop-start switchover rules run in `delivery`, which also has API access. The `secrets` example models the Secret Store (Vault) itself, whose one `Process` may stop serving during an update and so runs under a stop-start switchover. The `knowledge-platform` example is the largest. The `edge` and `observability` examples provide the proxies and collectors the other examples look up. Every resolution also computes provenance, the `Revision`, the path plan and egress rules.

4.1.1 Revision Evidence

An `Application`'s `Revision` is the hash of its canonical JSON representation. This includes its `Process`es with image, user, deadline, probes, placement, secrets, volumes, backups, environment and edges; its `Exposure`s and routes; its `Release Gate`, the service that holds the switch until every `Process` is healthy; and its migration.

The transformation builds that text from its own `Resolved Deployment` (Section 3.6), so one different value, one extra environment variable or one different order changes the digest. Ten `Application`s across five examples have a `Revision` in an expected file, and all ten match.

For these `Application`s, one digest comparison checks nearly every derived value in the element. A further test checks that the digest responds to deployment decisions: increasing memory in `minimal` from 256Mi to 512Mi changes the `Revision`, whereas changing only the capture time of the cluster-state snapshot leaves it unchanged.

4.1.2 Generated-File Evidence

An `Application`'s `Revision` excludes network policies, Vault documents and file paths. Comparing generated files checks these parts of the output as well: an incorrect allowed peer or destination path would fail the comparison even if the `Revision` matched. Across five examples and the cluster-wide output, all 167 committed files are reproduced byte-for-byte, including the final-pass job files.

4.1.3 Gaps in the Evidence

Two examples, `edge` and `observability`, have no `Revision`, no committed generated files and no expected model. Their `dependencies.json` files list no dependency edges. Evidence for these examples is therefore

limited to successful execution and the comparisons for other examples that use their proxies and collectors. Those comparisons check the values consumed by other projects, but leave the remaining proxy and collector configuration unchecked.

The expected `secrets` model was obtained by reviewing the generated output rather than constructing an independent oracle. It detects later changes to that output, but agreement does not independently establish correctness. The review found a gap in the specification: the Secret Store network policy allows no inbound traffic, although holders of secrets are allowed to send traffic to it. If enforced, the receiving policy would still block those connections; permission on the sending side is insufficient.

The hand-written model of `minimal` from Task 1 was updated after hand-in to follow the metamodel changes of Appendix A.2. Its `Revision` was copied from the oracle, so that field no longer provides an independent check.

4.1.4 Defects Found

Comparisons found a defect no single example had shown: the path plan placed cluster-wide route files under `apps/edge/`, which is also the directory of the `Project` named `edge`. The `edge` `Project`'s files would have mixed with routes from every other `Project`. The fix moved all cluster-wide files under `estate/`, in the specification's path rules and in the path plan, and the committed files now hold each file at its correct path.

4.2 Limitations

Agreement with the oracles supports the transformation's conformance to their expected outputs. It does not establish that the oracles or the specification are correct. Only the hand-written model of `minimal` was constructed directly from the specification.

The eight examples exercise every derivation rule, but a matching result for one input does not prove that every branch of that rule works. The `knowledge` example, which models a `Project` being retired, is not one of them; its successor `knowledge-platform` is.

Most cross-document checks have moved into OCL constraints. The remaining checks abort the transformation at the first problem and report no source location. The cross-document constraints still use `allInstances()`; the planned change to navigate from a single root object is not yet implemented.

QVTo caches mapping results but not query results. Lookups such as `edgeOf` and `Surface` address resolution are therefore repeated wherever they are used. Several queries scan all `Processes` for each `Process`, giving quadratic growth in work. For much larger inputs, measurements would be needed to identify queries worth replacing with cached mappings or precomputed properties.

Placement produces a list of eligible nodes for each `Process`; Kubernetes chooses the actual node. Each eligibility check considers that `Process` in isolation, so several may be eligible for a node even though their combined requests exceed its capacity.

When two `Processes` provide the same `Surface`, name order determines which is selected. Ordering between `ReconcileUnits` is defined at `Project` granularity because Flux orders these groups of files as whole units; individual `Applications` within a unit cannot be ordered separately.

The transformation computes provenance digests from the EMF representation of the inputs. This differs from the representation specified for the oracles, so those digests do not match. The digests used in generated files do match: the `Revision`, configuration-file name digests and the Vault policy job name.

Automated tests use the standalone executor. The Eclipse launch configuration and `plugin.xml` registration have only been checked manually.

4.3 Reflection

QVTo handled the two main kinds of reference used by the transformation. OCL queries searched all projects to find dependency providers; the mapping trace linked routes to the resolved `Process`es already created for them. This avoided maintaining a separate lookup table. The caller could also control when each operation ran, creating processes before their routes and computing revisions last. Repeated lookups were left unoptimised because performance was outside the scope of this proof of concept.

The greatest effort went into reproducing the `Revision`. The target model has a different structure from the specification's JSON, so the transformation had to reconstruct the specified field names, omissions and ordering before hashing. That reconstruction occupies about one sixth of `resolution.qvto`. A hash mismatch could mean either a wrong deployment decision or a wrong JSON representation, with no indication of where they differed. A better test design would compare that JSON directly before checking its digest, separating derivation errors from serialization errors. Using a Java hash function saved little of this work: most of the code was needed to construct the JSON in OCL.

Test evidence should also have been planned alongside the examples. `secrets` and `knowledge-platform` could run successfully before their output comparisons were added; execution alone did not reveal which derived values were wrong. Each example should have an expected result before it is used as evidence. Copying the oracle's `Revision` into the adapted Task 1 model meant that field could no longer expose a mistake in the oracle. Retaining the original model and documenting the adapted fields would make that loss of independent evidence explicit.

A Changes to the Task 1 Metamodels

This appendix lists the changes between the Task 1 metamodels and those the Task 2 transformations read and write. The comparison checked each class and feature in the `.ecore` files.

A third metamodel for the `Pinned input set`, the read-only documents no `Project` author writes, was added (Appendix A.1). The target metamodel gained derived values and twelve kinds of generated file (Appendix A.2), and the source metamodel gained declarations the other examples need (Appendix A.3). Four grammars were added (Appendix A.4).

No class was removed. The Task 1 hand-in showed OCL helper definitions such as `levelsAbove` and `derivesDatabase` as derived attributes in the diagram. They remain OCL definitions in `project-intent.ocl` and were never Ecore features.

A.1 The Pinned Inputs Metamodel

The `pinnedinputs` package models the additional documents read by the resolution (Section 1.3).

The node contract (the list of machines and their capacity) is generated from the platform team's machine descriptions. Other programs publish the images lock (each image pinned to a digest) and the cluster-state snapshot. A `Project` build writes its migration proof (the schema version its migration was tested against).

A committed descriptor checks the source metamodel's structure against the specification's authored-document schemas. Pinned inputs are kept in a separate metamodel because they are not part of those schemas. The target metamodel has no references to pinned-input classes: the `Resolved Deployment` records the values derived from these inputs and their digests instead.

Figure 3 shows the metamodel. Table 8 shows which rules read each class.

Class	Read by
`NodeContract`, `Node`, `Allocatable`, `Gpu`, `Disk`	`eligible` compares resource requests with available capacity and checks hardware requirements. CPU quantities such as 53500m denote 53.5 cores.
`ImagesLock`, `LockedImage`	`locked`: every image of a `Process`, `Sidecar`, backup method, migration runner and Vault policy job.
`ClusterState`, `Binding`	`boundOf`: the machine that already holds a `Volume`.
`CurrentPlacement`, `RollbackBackup`	Retained as snapshot data for delivery operations; these classes do not affect the resolution's output.
`MigrationProof`, `ProvenApplication`	`migrationOf` records the tested schema version; `gate` also reads whether the release contains non-transactional changes.
`AssetFile`	`Asset::resolvedFor` reads the configuration-file text. Each file is wrapped in an `AssetFile`, rather than parsed with an Xtext grammar.

Table 8: The Classes of the Pinned Inputs Metamodel and the Rules That Read Them

Disk medium and architecture are strings in the node inventory, although the source metamodel restricts `Process` requests to enumerations. The inventory can describe hardware, such as an sdcard, that no `Process` may request. Reusing the source enumeration would prevent such nodes from being represented. Eligibility therefore compares each requested enumeration's spelling with the corresponding inventory string.

A.2 Changes to the Target Metamodel

Both halves of the target metamodel grew: the values the resolution derives for each `Application`, and the generated-file entries of the path plan. Derivation rules read policies from the `Platform Intent`; the target metamodel records the resulting values.

The root class is unchanged, but there is now one instance per `Project` instead of one for the whole cluster (Section 1.3).

Derived Values. Figure 4 shows every derived-value class that is new or changed. Table 9 gives the reason for each.

Task 1 described many values in prose but modelled only those needed by its worked example. The other

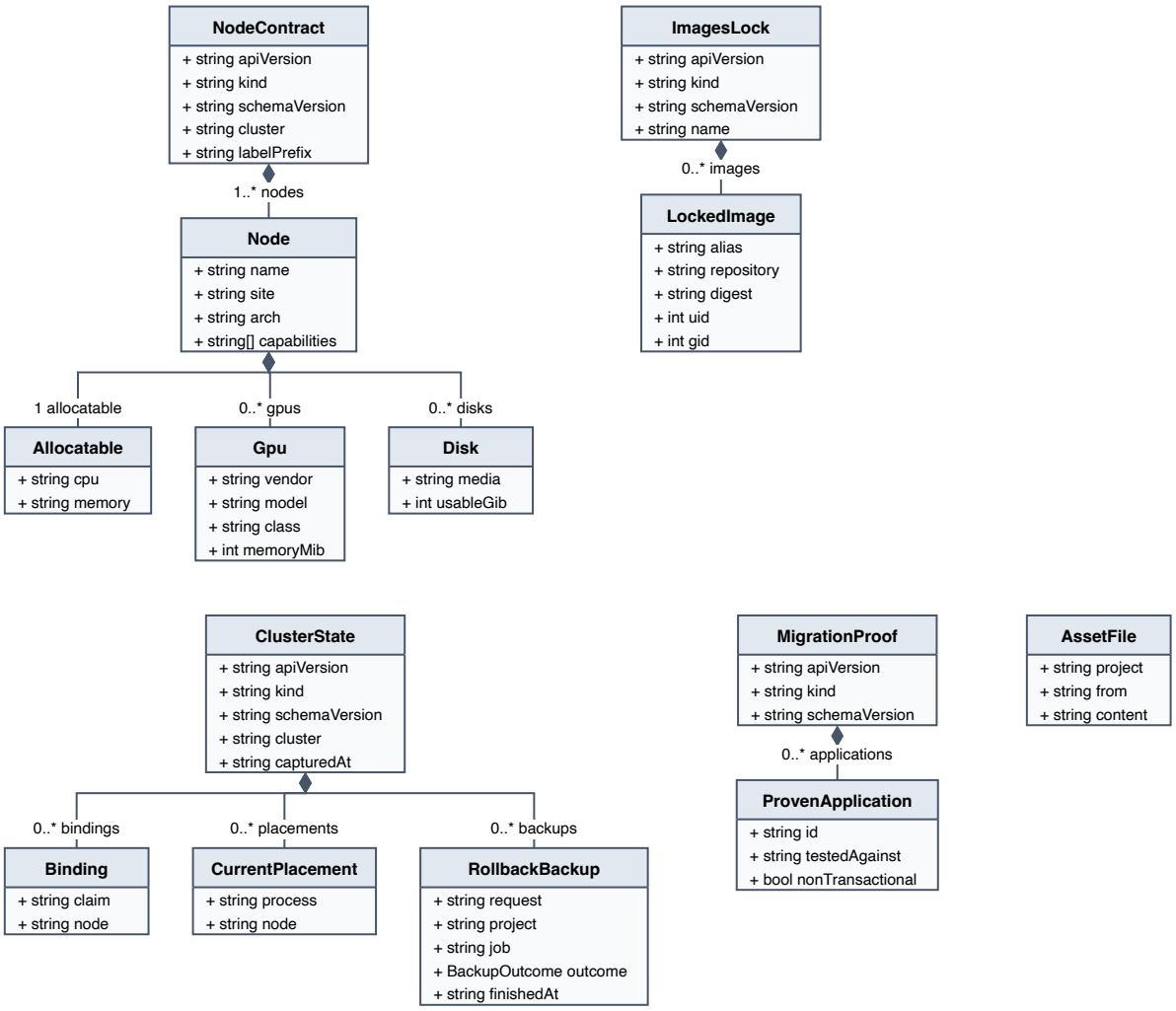


Figure 3: The Pinned Inputs Metamodel. Every class is new in Task 2

examples also need migrations, backups with database connections, secrets synchronised into the cluster, API access, `Sidecar`s, configuration files, and two cluster-wide values: the Vault policy job, which writes every Vault policy, and the route of Flux's events.

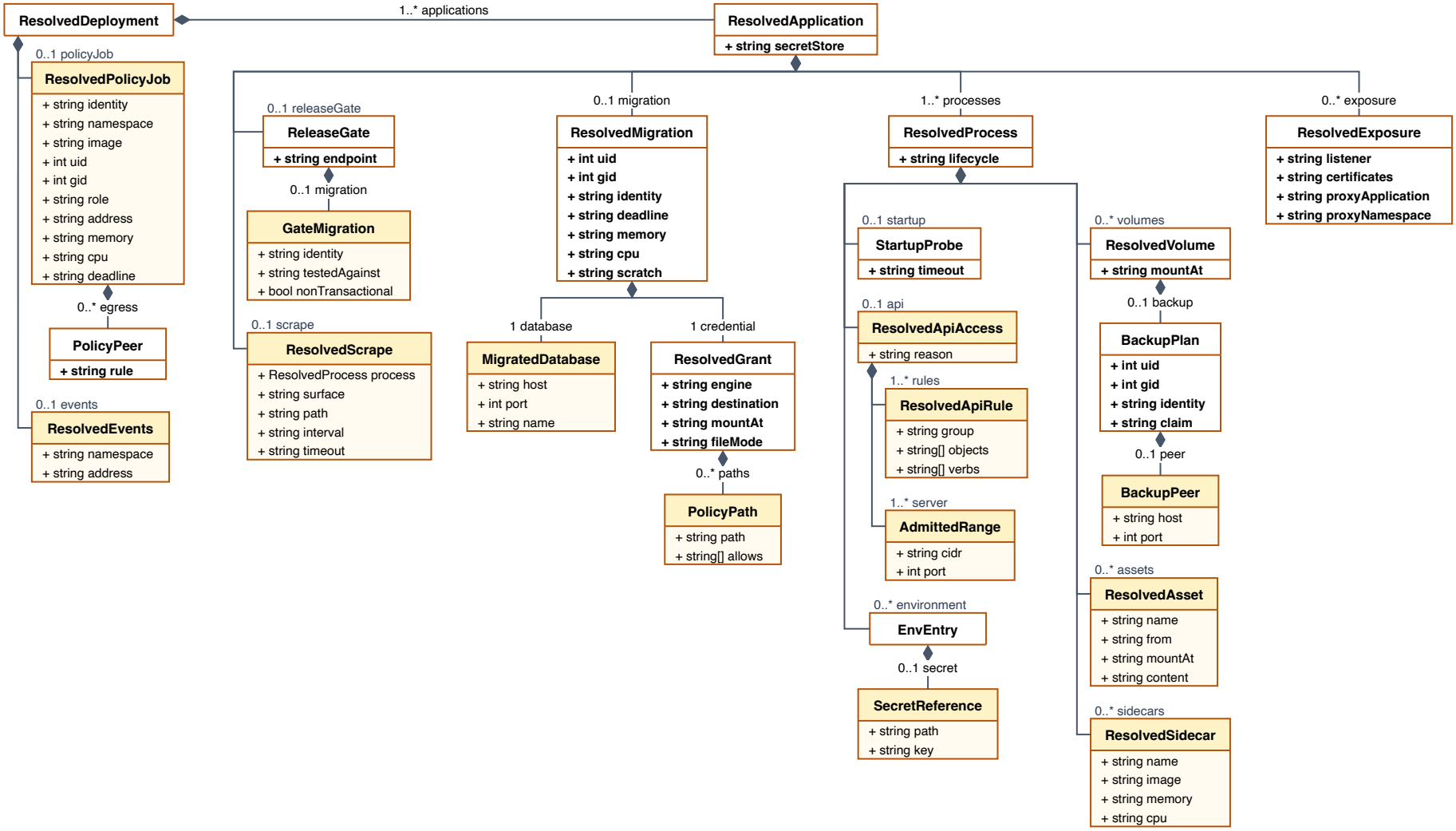


Figure 4: Derived-Value Classes of the Target Metamodel That Changed. A filled class is new in Task 2; a white one is a Task 1 class and shows only the features Task 2 added, in bold. A diamond marks containment; a class contained in several places is drawn under its first container only

Class	Change	Reason
`ResolvedDeployment`	added `policyJob`, `events`	The Vault policy job and the route for Flux's events belong to the whole cluster; every `Project` model carries the same job; the final pass plans their files.
`ResolvedPolicyJob`, `ResolvedEvents`	new	The job's service account (`identity`), image, Vault role, resources and egress; the events receiver's namespace and address.
`ResolvedApplication`	added `secretStore`, `scrape`	The Secret Store (Vault) address the operator syncs from, and how the metrics stack scrapes the `Application`, which the revision element names.
`ResolvedScrape`	new	The scraped `Process`, `Surface` (named port), path, interval and timeout.
`ReleaseGate`	added `endpoint`, `migration`	An address that every `Canary` (Flagger) webhook calls; the migration the gate starts and may undo.
`GateAnalysis`	added `errorRate`, `latency`	The thresholds of the two analysis checks, from the `Platform Intent`.
`GateMigration`	new	The migration's service account and what its proof records.
`ResolvedMigration`	added `uid`, `gid`, `identity`, `deadline`, `memory`, `cpu`, `scratch`, `database`, `credential`, `egress`	Task 1 named the service account, owner role and deadline in prose. A migration job also needs the database address, the owner credential that only it holds, and a network policy.
`MigratedDatabase`	new	The database host, port and name.
`ResolvedProcess`	added `lifecycle`, `api`, `assets`, `sidecars`	Whether the `Process` serves, runs once or prepares (which the revision element names); its API access, configuration files and `Sidecar`s.
`ResolvedApiAccess`, `ResolvedApiRule`, `AdmittedRange`	new	The API rules a `Process` holds; the address range a policy admits: the API server for API access, the backup off-cluster destination.
`ResolvedAsset`, `ResolvedSidecar`	new	A configuration file with a content-derived name; a `Sidecar` with a locked image and resources.
`ResolvedGrant`	`path` and `access` optional; added `engine`, `paths`, `destination`, `mountAt`, `fileMode`	A transit or database `Secret` has no single path; it is recorded as the Vault paths its policy covers. A synced `Secret` needs a Kubernetes `Secret` copied to its mount.
`PolicyPath`	new	One path of a Secret policy and the operations allowed on it.
`ResolvedVolume`	added `mountAt`	The workload template needs the mount path.
`BackupPlan`	added `user`, `group`, `identity`, `claim`, `egress`, `peer`, `credential`, `destinations`	A backup runs as its own service account with its own claim and network policy; it connects to the `Process` it dumps and copies off-cluster with a credential.
`BackupPeer`	new	Where a backup connects, and the credential it logs in with.
`EnvEntry`	`value` optional; added `secret`	A secret placeholder becomes a reference, so the value never appears in a file.
`SecretReference`	new	The secret path and key a variable reads.
`StartupProbe`	added `timeout`	The specification gives the startup probe the `Platform Intent` probe timeout.

Class	Change	Reason
`ResolvedExposure`	added `listener`, `certificates`, `proxyApplication`, `proxyNamespace`	The entry point's protocol and certificate source; the proxy whose middleware the routes name.
`PolicyPeer`	added `rule`; `process` optional	Which rule admitted the peer (whom a network policy lets each `Process` talk to), so two rules admitting one peer stay two entries, as the oracles record them; a peer such as cluster DNS has a namespace but no `Process`.

Table 9: Changes to the Derived-Value Classes of the Target Metamodel

Generated Files. Figure 5 shows every subclass of `Deliverable`, the generated-file entry, up from Task 1's nine to 21.

Workload, Canary and service account files, which Task 1 tied to one `Process`, now list all of an `Application`'s `Processes`, because the specification puts one file per `Application` holding all its `Process` resources. For the same reason, the network policy file holds one `ProcessPolicy` per `Process`, and the Secret file refers to its `Application` and holds one `SecretHolder` per service account, since a backup's service account holds Secrets too. Task 1's worked example had one `Process`, so this difference did not show.

`MonitorFile` gained timeout and kind: a blue/green `Process` is scraped through its Pods; any other through its `Service`.

The twelve new file kinds are Services, volume claims, configuration maps, backups, disruption budgets, migrations, RBAC, the secret operator connection, Vault policies, Vault roles, the Vault policy job and the events route.

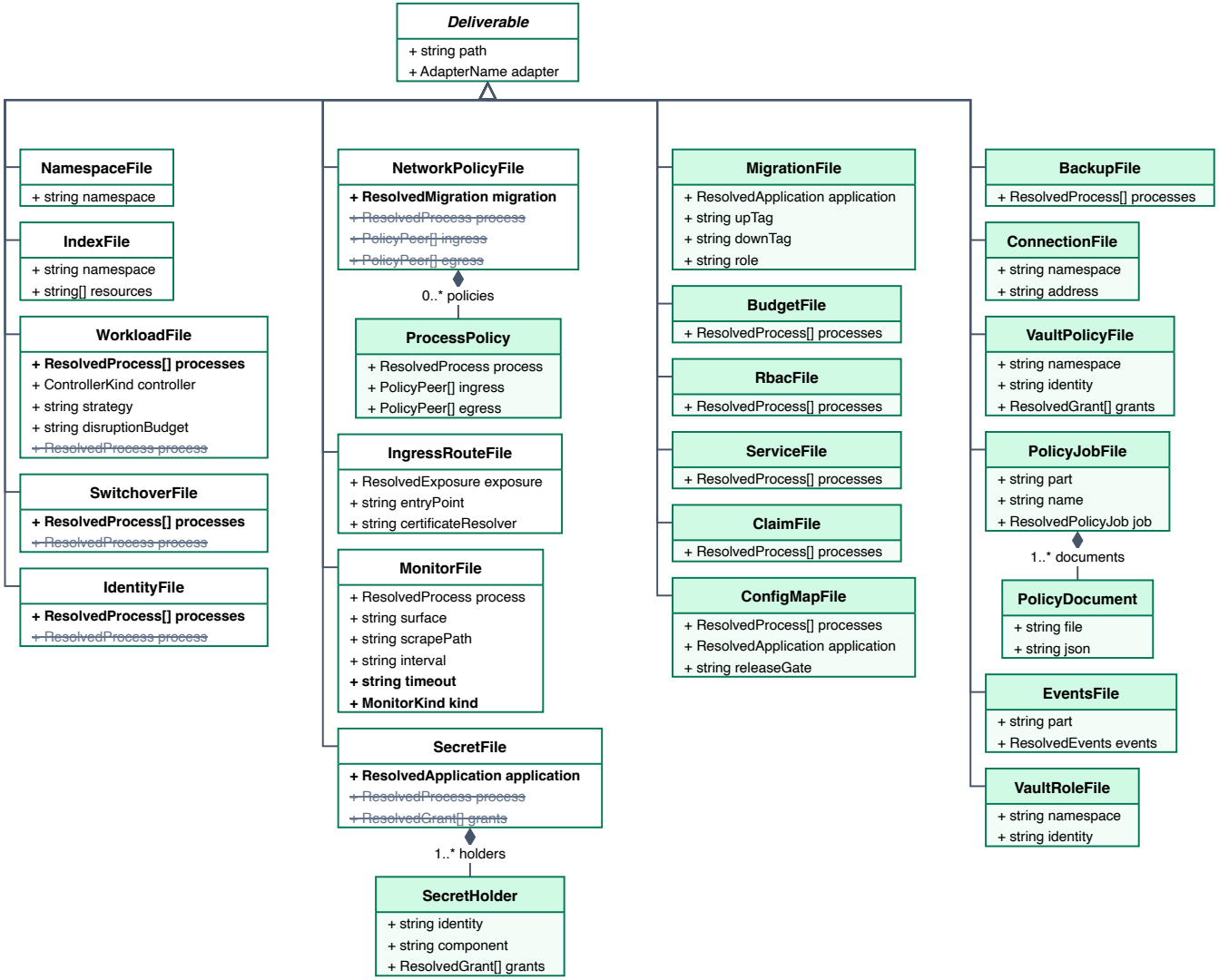


Figure 5: The Generated-File Classes of the Target Metamodel, Task 1 Against Task 2. A filled class is new in Task 2; in a white Task 1 class, bold marks a feature Task 2 added and strikethrough one it removed

The hand-written model of `minimal` from Task 1 was updated with these changes. Its files refer to the `Process` through `processes`, it gained the configuration map that carries the `ReleaseGate` inputs, and its `Revision` was set to the oracle's (Section 4.1.3).

A.3 Changes to the Source Metamodel

The source metamodel kept every class. Table 10 lists what was added or changed.

Most additions make platform policies explicit that Task 1 left implicit. Declaring these values in the `Platform Intent` keeps them out of the transformation rules, where they would otherwise be hard-coded.

Class	Change	Reason
`Process`	added `api` (`ApiAccess`, `ApiRule`, `ApiVerb`)	The `Process` declares the Kubernetes API access it needs, with a reason; the platform's own delivery `Applications` need it.
`Platform`	added `apiAccess` (`ApiAccessPolicy`)	Which `Applications` may hold API access, and the API server address range.
`Platform`	added `secretStore`, `telemetry` (`TelemetryPolicy`), `policyJob` (`VaultPolicyJob`)	References to the Secret Store, the telemetry collector and the metrics stack, whose addresses the resolution finds in their `Projects`; Vault policy job image, role and resources.
`DeliveryPolicy`	added `gate`, `reporter`	References to the `Release Gate` and the events receiver `Applications`.
`AnalysisPolicy`	added `errorRate`, `latency`	The thresholds the `Release Gate` checks.
`Substrate`	added `clusterDns`	The namespace every egress policy admits for DNS.
`EnginePolicy`	added `surface`	The port a database engine backup connects to.
`OffClusterCopy`	added `egress` (`DestinationRange`)	The address ranges a backup copy goes to.
`HandoverLedger`	added `serving` (`ServingLocation`)	Where a `Project` not yet moved still runs, so an edge to it resolves.
`RenderedArtifacts`	added `pullSecret`	Used by delivery, not by the resolution.
`Placeholder`	added `suffix`	Text after a placeholder, as in `${exposure:auth.public#url}/login`.
`Grant`	`rotation` required	Every `Grant` states how it tolerates rotation, which decides its restart targets.
`Volume`	`size` required	A claim cannot be generated without a size.
`EffectiveProject`	removed `apiVersion`, `kind`, `schemaVersion`	The flattened model is written by the flattening, not by an author, so it has no document header.

Table 10: Changes to the Source Metamodel

A.4 Grammars for the Pinned Inputs

The pinned inputs use grammars rather than a Java YAML reader or hand-written XMI, so the transformation reads the examples' YAML files unchanged and its results can be compared with their oracles.

Four grammars were added beside the Project and Platform grammars: NodeContract, ImagesLock, ClusterState and MigrationProof. Each inherits the Project grammar, reusing its terminals, indentation tokens and scalar rule, and adds only the rules for its own document. As with the `Platform Intent`, the file name selects the grammar: `node-contract.yml`, `images.lock.yml`, `cluster-state.yml` and `migration-proof.yml`.

One input file changed: the Project grammar scalar terminal cannot contain a colon, so the snapshot's capture time is now quoted, as digests and URLs already were. The value is the same either way.

Example 5 shows the smallest grammar.

```
grammar dev.jorisjonkers.deploykit.emf.syntax.ClusterState
        with dev.jorisjonkers.deploykit.emf.syntax.ProjectIntent
import "https://jorisjonkers.dev/deploy-kit/pinned-inputs/1"

ClusterState returns ClusterState:
        (('apiVersion' ':' apiVersion=Text)
        & ('kind' ':' kind=Text)
        & ('schemaVersion' ':' schemaVersion=Text)
        & ('cluster' ':' cluster=Text)
        & ('capturedAt' ':' capturedAt=Text)
        & ('bindings' ':' BEGIN (DASH BEGIN bindings+=Binding END)* END)
        & ('placements' ':' BEGIN (DASH BEGIN placements+=CurrentPlacement END)* END)
        & ('backups' ':' BEGIN (DASH BEGIN backups+=RollbackBackup END)* END)?);

Binding returns Binding:
        (('claim' ':' claim=Text)
        & ('node' ':' node=Text));
```

Example 5: The Grammar of the Cluster-State Snapshot, Abridged

B The Hand-in Archive

The archive holds `report.pdf`, a `README.md` with the steps from Appendix B.1, and two folders.

`task-2-transformations/` is an Eclipse project for reading the models (Table 11), imported via `File > Import > Existing Projects into Workspace`. It contains no Java, so the transformations do not run from it: the resolution and the final pass call the `Hashing` library, which the implementation plug-in registers.

`implementation/` holds the EMF implementation and the examples the tests read.

Path	Contents
`model/`	The source, pinned-inputs and target metamodels.
`transforms/`	`lower.qvto`, `resolution.qvto` and `estate.qvto`.
`examples/platform/`	The `Platform Intent` and the pinned inputs every example reads.
`examples/<example>/authored/`	The `Project Intent` with its env and configuration files.
`examples/<example>/resolution.intent.xmi`	The intent extent: authored and flattened `Projects`, plus `Platform Intent`.
`examples/<example>/resolution.pinned.xmi`	The pinned extent.
`examples/<example>/<project>.resolveddeployment`	The `Resolved Deployment` the transformation wrote.
`examples/<example>/dependencies.json`	The edges exported from it, beside `expected-dependencies.json`, a copy of the oracle `expected/dependencies.json`.
`examples/minimal/minimal.hand-written.resolveddeployment`	The hand-written target model from Task 1, updated in Appendix A.2.
`examples/secrets/secrets.expected.resolveddeployment`	The complete expected model of `secrets` (Section 4.1.3).

Table 11: The Models Project of the Archive, One Folder per Example of Table 7

Every XMI declares its metamodel in `xsi:schemaLocation`, so Eclipse opens it in the Sample Reflective Ecore Model Editor without a registered package.

B.1 Running the Transformations

1. In `implementation/emf`, run `./mvnwcleanverify` with JDK 21. It builds the implementation, runs every test, and writes each example's models and edges under `bundles/cli/target/parity/<example>/`. It takes about four minutes.
2. In Eclipse Modeling Tools with the QVT-Operational SDK, import `implementation/emf` via `File > Import > Existing Maven Projects`.
3. Run `resolution.launch` in `dev.jorisjonkers.deploykit.emf.resolve`. It resolves `minimal` from the extents step 1 wrote.

References

[1] P. Merle and F. Petrillo. Visualizing Cloud-native Applications with KubeDiagrams. In *Proc. VISSOFT 2025*, pp. 106–116, 2025. Tool: https://github.com/philippemerle/KubeDiagrams.

[2] KubeOrch, visual Kubernetes workflow orchestrator. https://github.com/KubeOrch (repositories `core` and `ui`), visited 15 October 2026.

[3] kubert, a kubectx and kubens alternative. https://github.com/idebeijer/kubert, visited 15 October 2026.

[4] L. Burgueño, J. Cabot and S. Gérard. The Future of Model Transformation Languages: An Open Community Discussion. *Journal of Object Technology* 18(3), 7:1–11, 2019.

[5] M. van Amstel, S. Bosems, I. Kurtev and L. Ferreira Pires. Performance in Model Transformations: Experiments with ATL and QVT. In *Proc. ICMT 2011*, LNCS 6707, pp. 198–212, 2011.

[6] Object Management Group. *Meta Object Facility Query/View/Transformation Specification*, version 1.3, 2016.